



자료구조 실습

12/10





목차

- 과제물 리뷰
- Graph Traversal란?
- DFS
- BFS
- 실행 결과
- Wrap-up for 기말고사



과제물 리뷰

- 스마트 냉장고 식재료 구조대
- 유통기한 순으로 정렬하면 비싼 고기를 나중에 먹다 버릴 수 있고, 가격 순으로 정렬하면 싼 야채가 다 썩어버림
- 현재 시점에서 가장 가치 손실이 크고 시급한 재료를 실시간으로 추천

$$\text{Priority} = \frac{\sqrt{\text{Price}}}{(\text{DaysLeft})^2}$$





과제물 리뷰

- 뉴욕 여행 방문 명소 우선순위
- 뉴욕 여행 중 제한된 시간에 최적의 명소를 방문하는 문제
- 실시간으로 여행자의 위치, 날씨, 시간 등을 고려한 추천 시스템



$$\text{priority} = (\text{인기도} \times 30) + (\text{날씨적합도} \times 20) + (\text{남은시간} \times 10) - (\text{거리} \times 5)$$



과제물 리뷰

- 심우주 통신 네트워크 다운링크 스케줄링
- 수많은 우주 탐사선이 동시에 지구로 통신을 요청하는 상황을 가정
- 통신 대역폭은 한정되어 있음
- 운용에서는 위성의 생존이 최우선

Priority

= 5000 * isEmergency // 생존 긴급 상황
+ 30 * bufferOccupancy // 내부 메모리 점유율
+ 10 * daysSinceContact // 통신 공백 기간





과제물 리뷰

- 폐기물 처리 우선 순위 시스템
- 매일 수많은 폐기물 발생
- 오염도나 유해성이 높은 폐기물은 빠른 수거 및 처리가 필요
- 경과 시간, 오염도, 재활용 가치, 유해성



```
priority = decayMinutes * 1 + wasteLevel * 50  
          + resourceValue * 30 + hazardLevel * 70;
```



과제물 리뷰

- 콘서트 우선순위 시스템
- ① 얼마나 오랜만의 내한인지
단독 콘서트 기준
- ② 공연장까지의 거리
경희대학교 국제캠퍼스 정문에서,
공연장까지의 거리를 계산
- ③ 티켓 가격
스탠딩 가격 기준



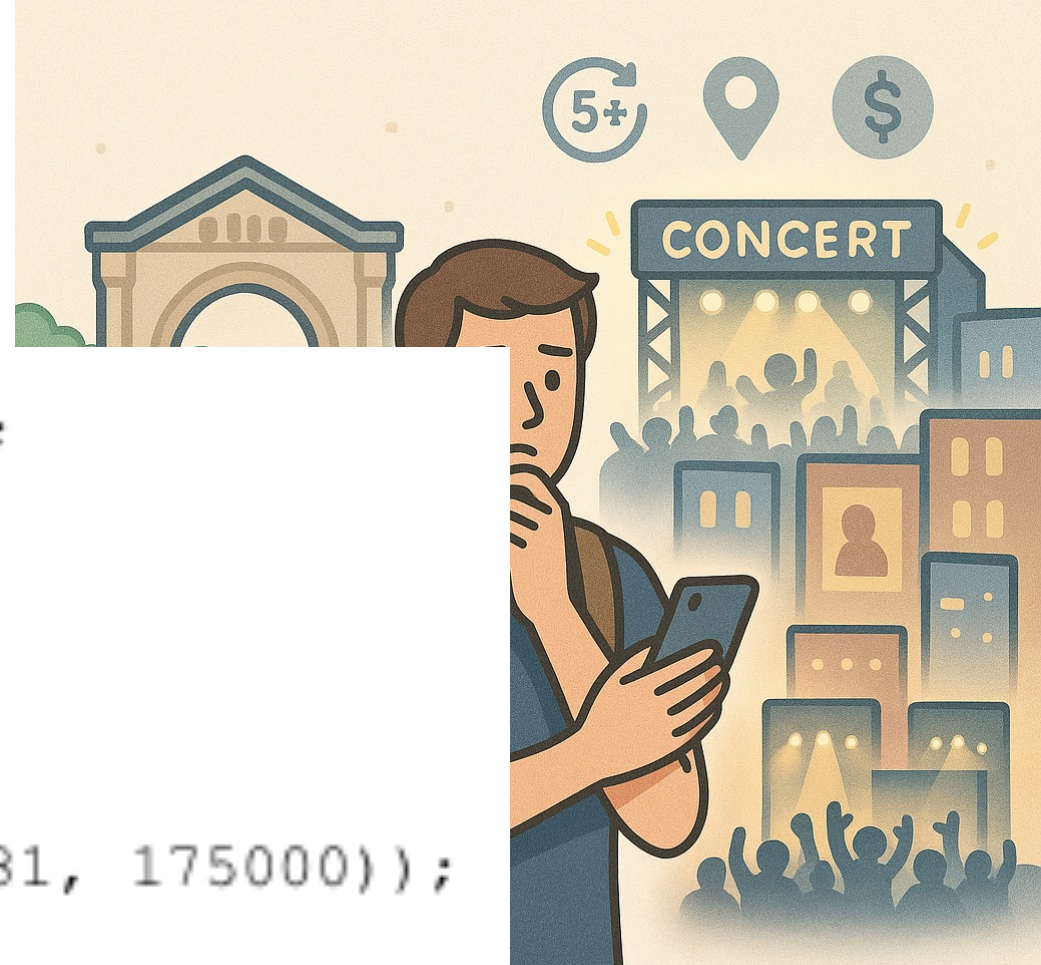
```
int calculatePriority(float period, int distance, int price) {  
    return period * 5 + (100 - distance) * 3 + price / 1000;  
}
```



과제물 리뷰

• 콘서트 우선순위 시스템

```
("Coldplay", 8, 73, 132000));  
("Oasis", 16, 73, 187000));  
("터치드", 0.3, 38, 154000));  
("자우림", 1, 50, 154000));  
("쏟애플", 0.5, 51, 132000));  
("Muse", 4.5, 56, 179000));  
("My Chemical Romance", 18, 81, 175000));  
("잔나비", 0.5, 37, 178000));  
("요아소비", 1, 84, 135000));  
("요네즈켄시", 1, 84, 132000));
```

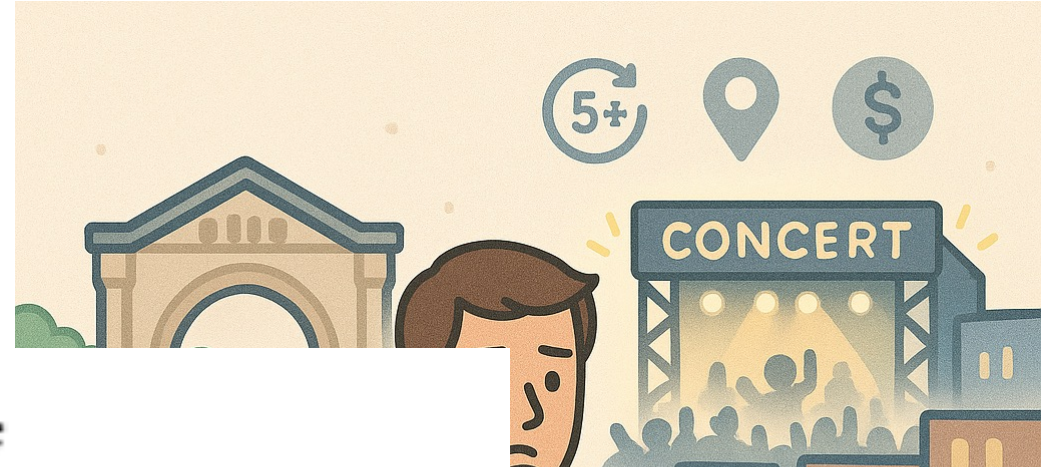


```
nce, int price) {  
+ price / 1000;
```



과제물 리뷰

• 콘서트 우선순위 시스템



```
("Coldplay", 8, 73, 132000));  
("Oasis", 16, 73, 187000  
("터치드", 0.3, 38, 154000(  
("자우림", 1, 50, 154000))  
("썸애플", 0.5, 51, 132000(  
("Muse", 4.5, 56, 179000  
("My Chemical Romance",  
("잔나비", 0.5, 37, 178000(  
("요아소비", 1, 84, 135000  
("요네즈켄시", 1, 84, 132000
```

가야할 콘서트 우선순위 정하는중...

--- 이걸 꼭 가야해! ---

Pop #1 → 잔나비 (priority = 367)

Pop #2 → Oasis (priority = 348)

Pop #3 → 터치드 (priority = 340)



과제물 리뷰

- 카카오 택시 배차 우선순위

$priority = waitMinutes * 3 + urgency * 40 + (vipLevel)^2 * 2 + blue * 30 - distance * 2$

- 지구 관측 위성의 관측 작업 순서 결정

$[20 \times importance + waiting - 25 \times consume - 2 \times multi - 50 \times full]$

- 독서 우선 순위 추천 큐

```
int priority = goalWeight(b.goalType) * 1000 // 목표 유형이 전체를 크게 갈라줌
+ deadlineScore * 20 // 마감 임박 책 가중치
+ remainingScore * 2 // 남은 분량 가중치
+ recencyScore; // 오랫동안 안 읽은 책 보너스
```



과제물 리뷰

• KBO 신인 드래프트 우선순위 시뮬레이터

priority =

batting Avg × 700 (공격 핵심 능력)

+ homerun × 4 (장타 가치)

+ (position==1 ? (300 - era×30) : 0) (ERA 낮을수록 고득점)

– age × 1.5 (어릴수록 미래 가치 높음)

– injury? 50 : 0 (부상 리스크 반영)





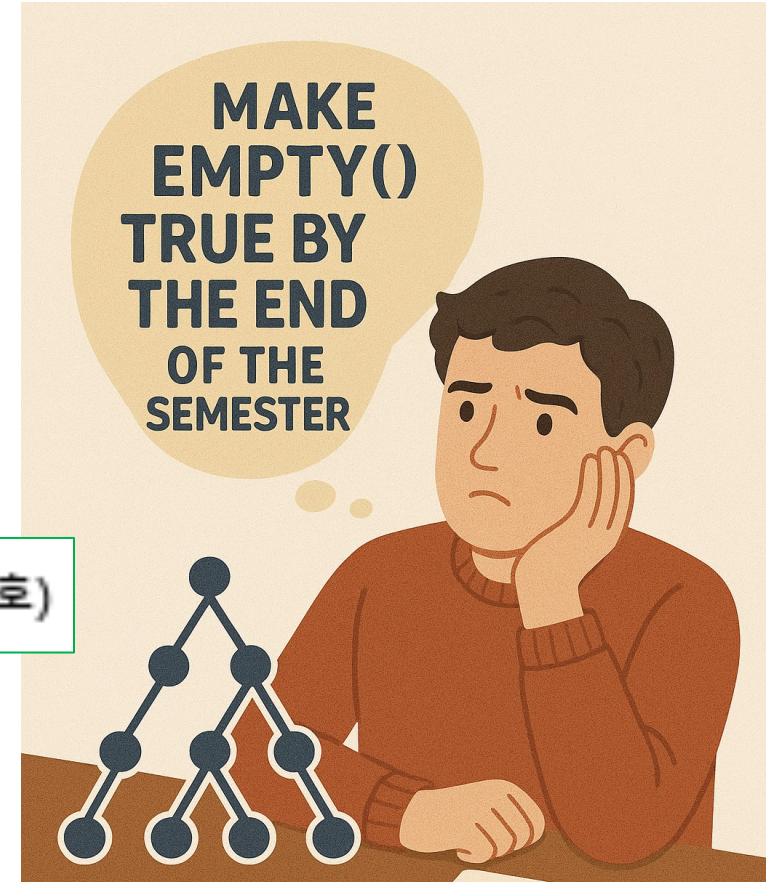
과제물 리뷰

- 과제 우선순위 프로그램

컴퓨터공학과는 과제가 참 많다. 오늘은 무슨 과제를 해야 할까?

- 과제가 추가될 때 마다 MaxHeap에 insert() 하여 가장 급한 과제를 출력
- 과제가 완료될 때 마다 pop()

Priority 계산식은 $100 \times (\text{종강날짜} - \text{마감날짜}) + (\text{최대 과제 수} - \text{과제번호})$





과제물 리뷰

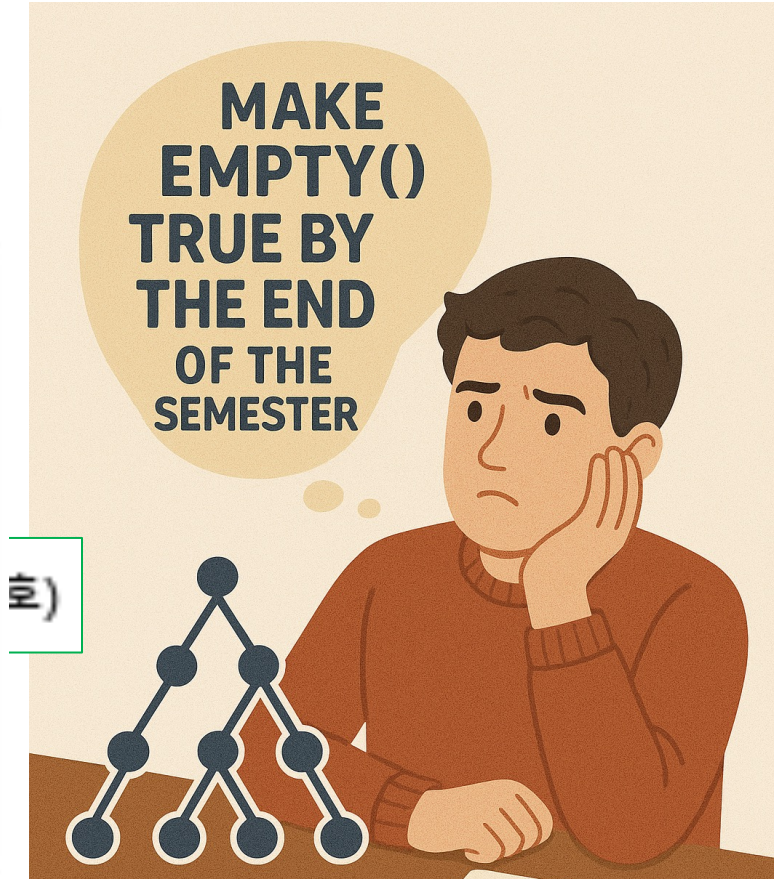
- 과제 우선순위 프로그램

컴퓨터공학과는 과제가

- 과제가 추가될
하여 가장 급한
- 과제가 완료될

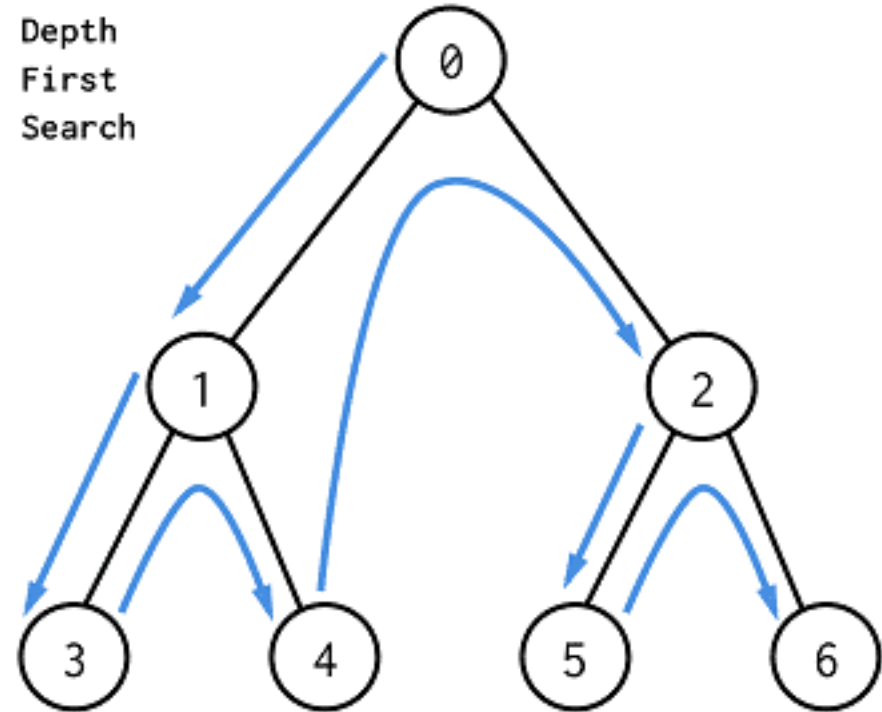
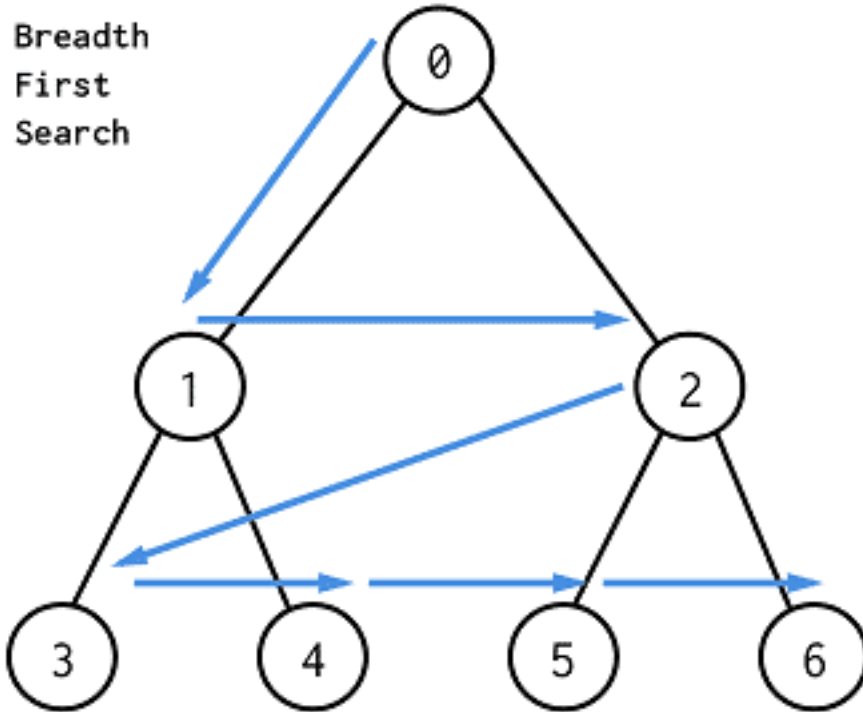
Priority 계산식은 $100 \times$

```
Microsoft Visual Studio 디버그 × + v
Pop#1 -> 인지프 #3 20148점
Pop#2 -> 딥러닝 #4 19146점
Pop#3 -> 인선대 #6 11947점
Pop#4 -> 인선대 #7 10544점
Pop#5 -> 딥러닝 #5 10143점
Pop#6 -> 인지프 #1 10050점
Pop#7 -> 논회 HW5 10045점
Pop#8 -> 인선대 #8 9642점
Pop#9 -> 자료구조 #1 1840점
```





Graph Traversal란?





BFS(Graph *g, int start)

```
void BFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int queue[MAX_VERTICES];
    int front = 0, rear = 0;

    if (start < 0 || start >= MAX_VERTICES) {
        printf("\n[BFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점을 큐에 enqueue
    queue[rear] = start;

    printf("\n[BFS] Start from %d\n", start);

    while (front < rear) { // 큐가 빌 때까지 ...
    }
}
```



BFS(Graph *g, int start)

```
void BFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int queue[MAX_VERTICES];
    int front = 0, rear = 0;

    if (start < 0 || start >= [REDACTED]) {
        printf("\n[BFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점을 큐에 enqueue
    queue[[REDACTED]] = start;

    printf("\n[BFS] Start from %d\n", start);

    while ([REDACTED]) { // 큐가 빌 때까지 ...
    }
}
```



BFS(Graph *g, int start)

```
void BFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int queue[MAX_VERTICES];
    int front = 0, rear = 0;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[BFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점을 큐에 enqueue
    queue[ ] = start;

    printf("\n[BFS] Start from %d\n", start);

    while ( ) { // 큐가 빌 때까지 ...
    }
}
```



BFS(Graph *g, int start)

```
void BFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int queue[MAX_VERTICES];
    int front = 0, rear = 0;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[BFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점을 큐에 enqueue
    queue[rear++] = start;

    printf("\n[BFS] Start from %d\n", start);

    while ( ) { // 큐가 빌 때까지 ...
    }
}
```



BFS(Graph *g, int start)

```
void BFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int queue[MAX_VERTICES];
    int front = 0, rear = 0;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[BFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점을 큐에 enqueue
    queue[rear++] = start;

    printf("\n[BFS] Start from %d\n", start);

    while ( ) { // 큐가 빌 때까지 ...
    }
}
```



BFS(Graph *g, int start)

```
void BFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int queue[MAX_VERTICES];
    int front = 0, rear = 0;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[BFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점을 큐에 enqueue
    queue[rear++] = start;

    printf("\n[BFS] Start from %d\n", start);

    while ( ) { // 큐가 빌 때까지 ...
    }
}
```

```
while ( ) { // 큐가 빌 때까지
    int v = queue[ ]; // dequeue

    if ( ) { // 아직 방문하지 않은 정점이면
        printf("Visit %d\n", v);
        = 

        // 인접한 정점들을 큐에 enqueue
        for (int i = 0; i < g->numVertices; i++) {
            if (g-> == 1 && ) {
                queue[ ] = 
            }
        }
    }
}
```



BFS(Graph *g, int start)

```
void BFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int queue[MAX_VERTICES];
    int front = 0, rear = 0;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[BFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점을 큐에 enqueue
    queue[rear++] = start;

    printf("\n[BFS] Start from %d\n", start);

    while (front < rear) { // 큐가 빌 때까지 ...
    }
}
```

```
while (front < rear) { // 큐가 빌 때까지
    int v = queue[front]; // dequeue

    if (visited[v] == 0) { // 아직 방문하지 않은 정점이면
        printf("Visit %d\n", v);
        visited[v] = 1;

        // 인접한 정점들을 큐에 enqueue
        for (int i = 0; i < g->numVertices; i++) {
            if (g->adj[v][i] == 1 && visited[i] == 0) {
                queue[rear++] = i;
            }
        }
    }
}
```



BFS(Graph *g, int start)

```
void BFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int queue[MAX_VERTICES];
    int front = 0, rear = 0;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[BFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점을 큐에 enqueue
    queue[rear++] = start;

    printf("\n[BFS] Start from %d\n", start);

    while (front < rear) { // 큐가 빌 때까지 ...
    }
}
```

```
while (front < rear) { // 큐가 빌 때까지
    int v = queue[front++]; // dequeue

    if ( ) { // 아직 방문하지 않은 정점이면
        printf("Visit %d\n", v);
        =

        // 인접한 정점들을 큐에 enqueue
        for (int i = 0; i < g->numVertices; i++) {
            if (g-> == 1 && ) {
                queue[ ] =
            }
        }
    }
}
```



BFS(Graph *g, int start)

```
void BFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int queue[MAX_VERTICES];
    int front = 0, rear = 0;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[BFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점을 큐에 enqueue
    queue[rear++] = start;

    printf("\n[BFS] Start from %d\n", start);

    while (front < rear) { // 큐가 빌 때까지 ...
    }
}
```

```
while (front < rear) { // 큐가 빌 때까지
    int v = queue[front++]; // dequeue

    if (!visited[v]) { // 아직 방문하지 않은 정점이면
        printf("Visit %d\n", v);
        visited[v] = 1;

        // 인접한 정점들을 큐에 enqueue
        for (int i = 0; i < g->numVertices; i++) {
            if (g->adj[v][i] == 1 && !visited[i]) {
                queue[rear++] = i;
            }
        }
    }
}
```



BFS(Graph *g, int start)

```
void BFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int queue[MAX_VERTICES];
    int front = 0, rear = 0;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[BFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점을 큐에 enqueue
    queue[rear++] = start;

    printf("\n[BFS] Start from %d\n", start);

    while (front < rear) { // 큐가 빌 때까지 ...
    }
}
```

```
while (front < rear) { // 큐가 빌 때까지
    int v = queue[front++]; // dequeue

    if (!visited[v]) { // 아직 방문하지 않은 정점이면
        printf("Visit %d\n", v);
        visited[v] = 1;

        // 인접한 정점들을 큐에 enqueue
        for (int i = 0; i < g->numVertices; i++) {
            if (g->adj[v][i] == 1 && !visited[i]) {
                queue[rear++] = i;
            }
        }
    }
}
```



BFS(Graph *g, int start)

```
void BFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int queue[MAX_VERTICES];
    int front = 0, rear = 0;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[BFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점을 큐에 enqueue
    queue[rear++] = start;

    printf("\n[BFS] Start from %d\n", start);

    while (front < rear) { // 큐가 빌 때까지 ...
    }
}
```

```
while (front < rear) { // 큐가 빌 때까지
    int v = queue[front++]; // dequeue

    if (!visited[v]) { // 아직 방문하지 않은 정점이면
        printf("Visit %d\n", v);
        visited[v] = 1;

        // 인접한 정점들을 큐에 enqueue
        for (int i = 0; i < g->numVertices; i++) {
            if (g->adj[v][i] == 1 && [redacted]) {
                queue[[redacted]] = [redacted];
            }
        }
    }
}
```



BFS(Graph *g, int start)

```
void BFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int queue[MAX_VERTICES];
    int front = 0, rear = 0;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[BFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점을 큐에 enqueue
    queue[rear++] = start;

    printf("\n[BFS] Start from %d\n", start);

    while (front < rear) { // 큐가 빌 때까지 ...
    }
}
```

```
while (front < rear) { // 큐가 빌 때까지
    int v = queue[front++]; // dequeue

    if (!visited[v]) { // 아직 방문하지 않은 정점이면
        printf("Visit %d\n", v);
        visited[v] = 1;

        // 인접한 정점들을 큐에 enqueue
        for (int i = 0; i < g->numVertices; i++) {
            if (g->adj[v][i] == 1 && !visited[i]) {
                queue[ ] = 
            }
        }
    }
}
```



BFS(Graph *g, int start)

```
void BFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int queue[MAX_VERTICES];
    int front = 0, rear = 0;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[BFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점을 큐에 enqueue
    queue[rear++] = start;

    printf("\n[BFS] Start from %d\n", start);

    while (front < rear) { // 큐가 빌 때까지 ...
    }
}
```

```
while (front < rear) { // 큐가 빌 때까지
    int v = queue[front++]; // dequeue

    if (!visited[v]) { // 아직 방문하지 않은 정점이면
        printf("Visit %d\n", v);
        visited[v] = 1;

        // 인접한 정점들을 큐에 enqueue
        for (int i = 0; i < g->numVertices; i++) {
            if (g->adj[v][i] == 1 && !visited[i]) {
                queue[rear++] = v
            }
        }
    }
}
```



BFS(Graph *g, int start)

```
void BFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int queue[MAX_VERTICES];
    int front = 0, rear = 0;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[BFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점을 큐에 enqueue
    queue[rear++] = start;

    printf("\n[BFS] Start from %d\n", start);

    while (front < rear) { // 큐가 빌 때까지 ...
    }
}
```

```
while (front < rear) { // 큐가 빌 때까지
    int v = queue[front++]; // dequeue

    if (!visited[v]) { // 아직 방문하지 않은 정점이면
        printf("Visit %d\n", v);
        visited[v] = 1;

        // 인접한 정점들을 큐에 enqueue
        for (int i = 0; i < g->numVertices; i++) {
            if (g->adj[v][i] == 1 && !visited[i]) {
                queue[rear++] = i;
            }
        }
    }
}
```



DFS(Graph *g, int start)

```
void DFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int stack[MAX_VERTICES];
    int top = -1;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[DFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점 스택에 push
    stack[ ] = ;

    printf("\n[DFS] Start from %d\n", start);

    while ( ) { // 스택이 빌 때까지 ...
    }
}
```



DFS(Graph *g, int start)

```
void DFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int stack[MAX_VERTICES];
    int top = -1;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[DFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점 스택에 push
    stack[++top] = [REDACTED]

    printf("\n[DFS] Start from %d\n", start);

    while ( [REDACTED] ) { // 스택이 빌 때까지 ...
    }
}
```



DFS(Graph *g, int start)

```
void DFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int stack[MAX_VERTICES];
    int top = -1;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[DFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점 스택에 push
    stack[++top] = start;

    printf("\n[DFS] Start from %d\n", start);

    while ( ) { // 스택이 빌 때까지 ...
    }
}
```



DFS(Graph *g, int start)

```
void DFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int stack[MAX_VERTICES];
    int top = -1;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[DFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점 스택에 push
    stack[++top] = start;

    printf("\n[DFS] Start from %d\n", start);

    while ( ) { // 스택이 빌 때까지 ...
    }
}
```

```
while ( ) { // 스택이 빌 때까지
    int v = stack[ ]; // pop

    if ( ) { // 아직 방문하지 않은 정점이면
        printf("Visit %d\n", v);
        ;

        // 인접한 정점들을 스택에 push
        // 스택이므로, 역순으로 넣으면 작은 번호부터 방문
        for (int i = g->numVertices - 1; i >= 0; i--) {
            if (g-> == 1 && ) {
                stack[ ] = ;
            }
        }
    }
}
```



DFS(Graph *g, int start)

```
void DFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int stack[MAX_VERTICES];
    int top = -1;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[DFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점 스택에 push
    stack[++top] = start;

    printf("\n[DFS] Start from %d\n", start);

    while (top != -1) { // 스택이 빌 때까지 ...
    }
}
```

```
while (top != -1) { // 스택이 빌 때까지
    int v = stack[top]; // pop

    if (visited[v] == 0) { // 아직 방문하지 않은 정점이면
        printf("Visit %d\n", v);
        visited[v] = 1;

        // 인접한 정점들을 스택에 push
        // 스택이므로, 역순으로 넣으면 작은 번호부터 방문
        for (int i = g->numVertices - 1; i >= 0; i--) {
            if (g->adjList[v][i] == 1 && visited[g->adjList[v][i]] == 0) {
                stack[++top] = g->adjList[v][i];
            }
        }
    }
}
```



DFS(Graph *g, int start)

```
void DFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int stack[MAX_VERTICES];
    int top = -1;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[DFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점 스택에 push
    stack[++top] = start;

    printf("\n[DFS] Start from %d\n", start);

    while (top != -1) { // 스택이 빌 때까지 ...
    }
}
```

```
while (top != -1) { // 스택이 빌 때까지
    int v = stack[top--]; // pop

    if ( ) { // 아직 방문하지 않은 정점이면
        printf("Visit %d\n", v);
        // 방문 처리

        // 인접한 정점들을 스택에 push
        // 스택이므로, 역순으로 넣으면 작은 번호부터 방문
        for (int i = g->numVertices - 1; i >= 0; i--) {
            if (g-> == 1 && ) {
                stack[ ] = 
            }
        }
    }
}
```



DFS(Graph *g, int start)

```
void DFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int stack[MAX_VERTICES];
    int top = -1;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[DFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점 스택에 push
    stack[++top] = start;

    printf("\n[DFS] Start from %d\n", start);

    while (top != -1) { // 스택이 빌 때까지 ...
    }
}
```

```
while (top != -1) { // 스택이 빌 때까지
    int v = stack[top--]; // pop

    if (!visited[v]) { // 아직 방문하지 않은 정점이면
        printf("Visit %d\n", v);
        [redacted]

        // 인접한 정점들을 스택에 push
        // 스택이므로, 역순으로 넣으면 작은 번호부터 방문
        for (int i = g->numVertices - 1; i >= 0; i--) {
            if (g->[redacted] == 1 && [redacted]) {
                stack[[redacted]] = [redacted];
            }
        }
    }
}
```



DFS(Graph *g, int start)

```
void DFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int stack[MAX_VERTICES];
    int top = -1;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[DFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점 스택에 push
    stack[++top] = start;

    printf("\n[DFS] Start from %d\n", start);

    while (top != -1) { // 스택이 빌 때까지 ...
    }
}
```

```
while (top != -1) { // 스택이 빌 때까지
    int v = stack[top--]; // pop

    if (!visited[v]) { // 아직 방문하지 않은 정점이면
        printf("Visit %d\n", v);
        visited[v] = 1;

        // 인접한 정점들을 스택에 push
        // 스택이므로, 역순으로 넣으면 작은 번호부터 방문
        for (int i = g->numVertices - 1; i >= 0; i--) {
            if (g->adj[v][i] == 1 && !visited[g->adj[v][i]]) {
                stack[++top] = g->adj[v][i];
            }
        }
    }
}
```



DFS(Graph *g, int start)

```
void DFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int stack[MAX_VERTICES];
    int top = -1;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[DFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점 스택에 push
    stack[++top] = start;

    printf("\n[DFS] Start from %d\n", start);

    while (top != -1) { // 스택이 빌 때까지 ...
    }
}
```

```
while (top != -1) { // 스택이 빌 때까지
    int v = stack[top--]; // pop

    if (!visited[v]) { // 아직 방문하지 않은 정점이면
        printf("Visit %d\n", v);
        visited[v] = 1;

        // 인접한 정점들을 스택에 push
        // 스택이므로, 역순으로 넣으면 작은 번호부터 방문
        for (int i = g->numVertices - 1; i >= 0; i--) {
            if (g->adj[v][i] == 1 && [redacted]) {
                stack[[redacted]] = [redacted];
            }
        }
    }
}
```



DFS(Graph *g, int start)

```
void DFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int stack[MAX_VERTICES];
    int top = -1;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[DFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점 스택에 push
    stack[++top] = start;

    printf("\n[DFS] Start from %d\n", start);

    while (top != -1) { // 스택이 빌 때까지 ...
    }
}
```

```
while (top != -1) { // 스택이 빌 때까지
    int v = stack[top--]; // pop

    if (!visited[v]) { // 아직 방문하지 않은 정점이면
        printf("Visit %d\n", v);
        visited[v] = 1;

        // 인접한 정점들을 스택에 push
        // 스택이므로, 역순으로 넣으면 작은 번호부터 방문
        for (int i = g->numVertices - 1; i >= 0; i--) {
            if (g->adj[v][i] == 1 && !visited[i]) {
                stack[ ] = 
            }
        }
    }
}
```



DFS(Graph *g, int start)

```
void DFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int stack[MAX_VERTICES];
    int top = -1;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[DFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점 스택에 push
    stack[++top] = start;

    printf("\n[DFS] Start from %d\n", start);

    while (top != -1) { // 스택이 빌 때까지 ...
    }
}
```

```
while (top != -1) { // 스택이 빌 때까지
    int v = stack[top--]; // pop

    if (!visited[v]) { // 아직 방문하지 않은 정점이면
        printf("Visit %d\n", v);
        visited[v] = 1;

        // 인접한 정점들을 스택에 push
        // 스택이므로, 역순으로 넣으면 작은 번호부터 방문
        for (int i = g->numVertices - 1; i >= 0; i--) {
            if (g->adj[v][i] == 1 && !visited[i]) {
                stack[++top] = v
            }
        }
    }
}
```



DFS(Graph *g, int start)

```
void DFS(Graph *g, int start) {
    int visited[MAX_VERTICES] = {0}; // 방문 여부 배열
    int stack[MAX_VERTICES];
    int top = -1;

    if (start < 0 || start >= g->numVertices) {
        printf("\n[DFS] 잘못된 시작 정점입니다.\n");
        return;
    }

    // 시작 정점 스택에 push
    stack[++top] = start;

    printf("\n[DFS] Start from %d\n", start);

    while (top != -1) { // 스택이 빌 때까지 ...
    }
}
```

```
while (top != -1) { // 스택이 빌 때까지
    int v = stack[top--]; // pop

    if (!visited[v]) { // 아직 방문하지 않은 정점이면
        printf("Visit %d\n", v);
        visited[v] = 1;

        // 인접한 정점들을 스택에 push
        // 스택이므로, 역순으로 넣으면 작은 번호부터 방문
        for (int i = g->numVertices - 1; i >= 0; i--) {
            if (g->adj[v][i] == 1 && !visited[i]) {
                stack[++top] = i;
            }
        }
    }
}
```



Main 함수 실행

```
initGraph(&g, 7);

// 예시 간선들
// 0 - 1 - 2 - 4 - 6
//      \- 3
// 2 - 5
addEdge(&g, 0, 1); // 0-1
addEdge(&g, 1, 2); // 1-2
addEdge(&g, 1, 3); // 1-3
addEdge(&g, 2, 4); // 2-4
addEdge(&g, 2, 5); // 2-5
addEdge(&g, 4, 6); // 4-6

printf("==== Graph Traversal Practice\n\n");
printAdjMatrix(&g);

// 시작 정점을 0으로 두고 DFS, BFS 실행
DFS(&g, 0);
BFS(&g, 0);
```

```
==== Graph Traversal Practice (DFS / BFS) ====

[Adjacency Matrix]
  0  1  2  3  4  5  6
0:  0  1  0  0  0  0  0
1:  1  0  1  1  0  0  0
2:  0  1  0  0  1  1  0
3:  0  1  0  0  0  0  0
4:  0  0  1  0  0  0  1
5:  0  0  1  0  0  0  0
6:  0  0  0  0  1  0  0

[DFS] Start from 0
Visit 0
Visit 1
Visit 2
Visit 4
Visit 6
Visit 5
Visit 3

[BFS] Start from 0
Visit 0
Visit 1
Visit 2
Visit 3
Visit 4
Visit 5
Visit 6
```



Wrap-up for 기말고사

- Linked list
- BST
- Graph
- Graph_traversal

 1105-linked-list

 1112-bst

 1126-bst

 1203-graph

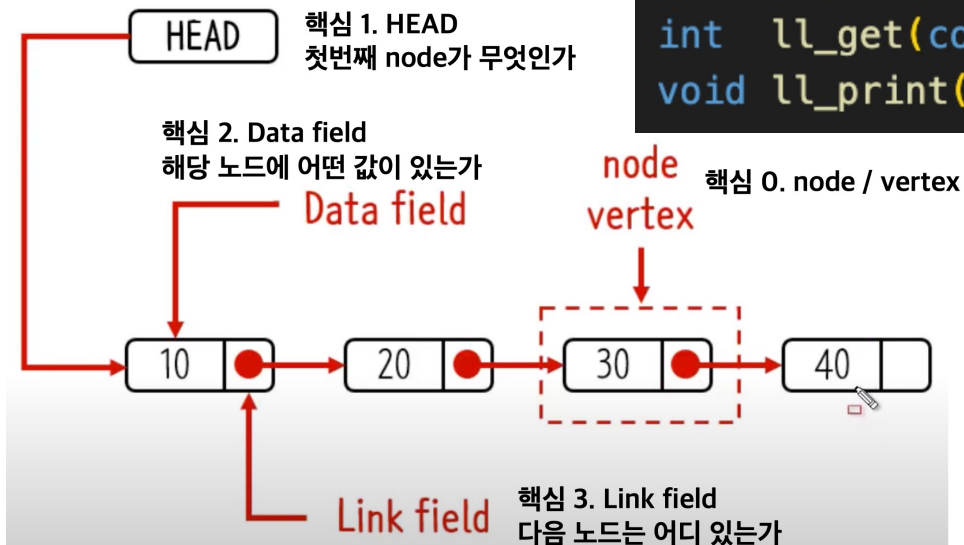
 1210-graph_traversal



Wrap-up for 기말고사

- Linked list

```
void ll_init(LinkedList* L);           // 리스트 초기화
int ll_is_empty(const LinkedList* L);   // 비었는지 확인
int ll_size(const LinkedList* L);       // 크기 반환
void ll_clear(LinkedList* L);           // 모두 삭제
void ll_push_front(LinkedList* L, int x); // 머리에 삽입
void ll_insert(LinkedList* L, int idx, int x); // idx 위치에 삽입
int ll_remove(LinkedList* L, int x);    // 값 x 첫 1개 삭제
int ll_get(const LinkedList* L, int idx); // idx번째 값 (잘못되면 -1)
void ll_print(const LinkedList* L, const char* tag); // 내용 출력
```



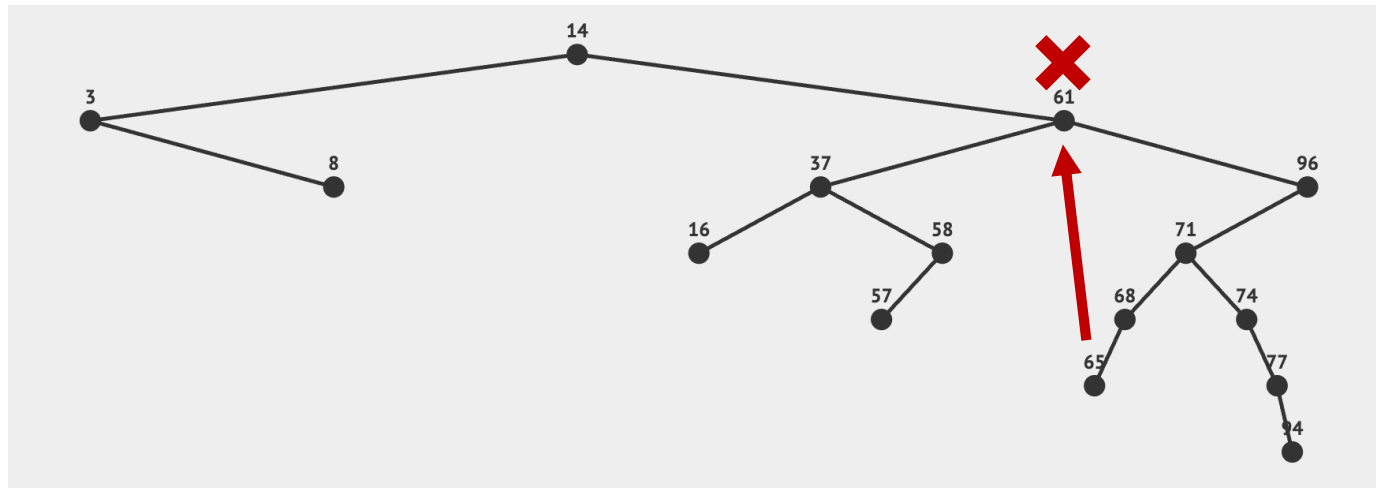


Wrap-up for 기말고사

- BST
 - 노드 검색
 - 노드 삽입
 - 노드 삭제
 - Preorder
 - Inorder
 - Postorder

노드 삭제

3번: 그 외



Let's remove 61

김민서 (imkmsh@khu.ac.kr)

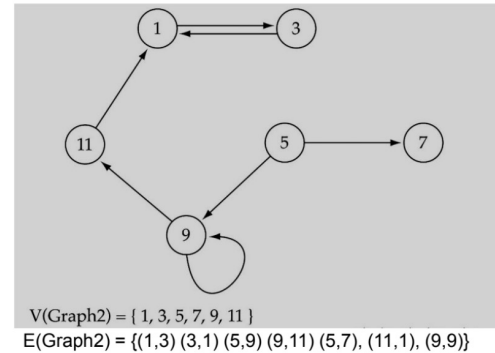
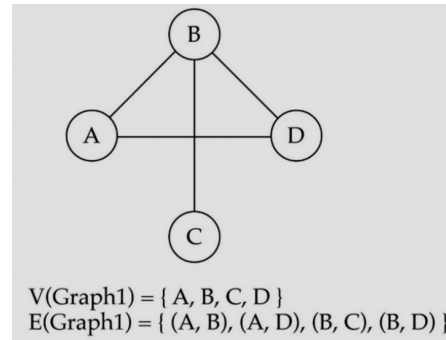
29



Wrap-up for 기말고사

- Graph
 - 인접 행렬, 인접 리스트 2가지 구현

- 정점(Vertex)과 간선(Edge)
- 방향성 여부 : 무방향(undirected) / 방향(directed)
- 가중치 여부 : 가중치 없는(unweighted) / 가중치 있는(weighted)



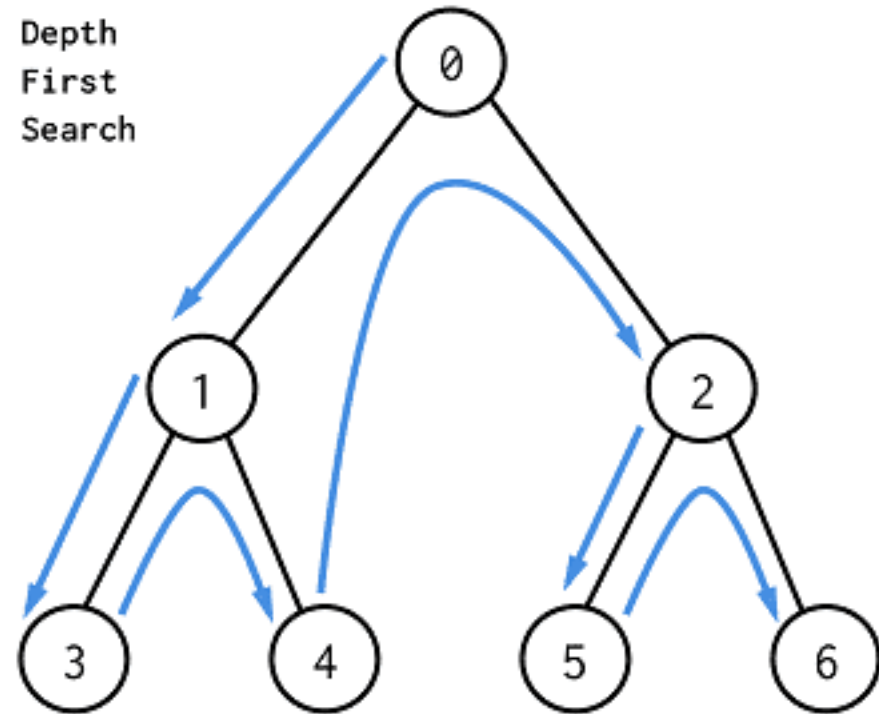
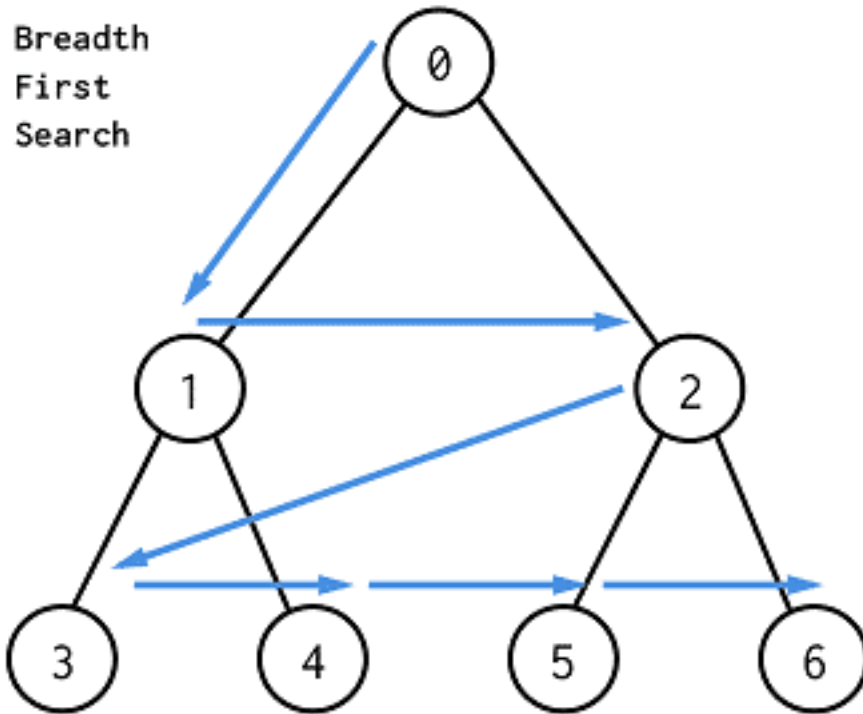
김민서 (imkmsh@khu.ac.kr)

2



Wrap-up for 기말고사

- Graph_traversal





25년 가을학기 수고하셨습니다

자료구조 실습 12/10

EOF